

Lecture Unit 8.1

More MPI features

Unit overview

- To implement manager-worker parallelism we will use some additional MPI features:
 - Wildcards
 - Groups and communicators

Wildcards

- In a manager/worker algorithm we have a manager process which receives requests from worker processes
- We don't know which worker will request work next so we need a way to receive a message from an unspecified source
- We can use the wildcards `MPI_ANY_SOURCE` to receive from an unspecified source
- Similarly `MPI_ANY_TAG` receives from an unspecified tag
- The rank and tag received from are returned in the `status` argument

MPI_Recv

- MPI_Recv: receive a message from another process
- Describes data to be received and connection to be made
- `int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)`

buf: receive buffer

count: number of items to receive

datatype: type of data

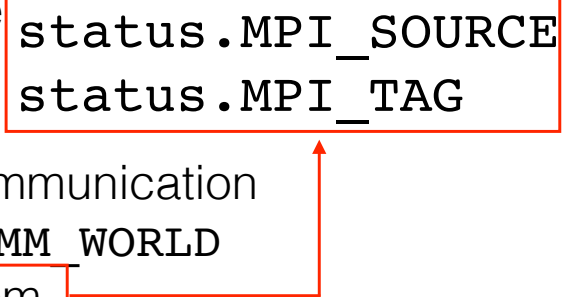
source: where to receive from

tag: user defined label for this communication

comm: communicator e.g. MPI_COMM_WORLD

status: rank and tag received from

`status.MPI_SOURCE`
`status.MPI_TAG`



wildcard.c

```
int main(int argc, char *argv[]){

    int myRank, nProcs;
    const int len=20;
    char message[len];
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
    MPI_Comm_size(MPI_COMM_WORLD, &nProcs);

    if (myRank==0){
        /* Receive one message from each process */
        for (int i=1; i<nProcs; i++){
            MPI_Recv(&message, len, MPI_CHAR, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
            printf("Received message: \"%s\" from rank %d with tag %d\n", message, status.MPI_SOURCE, status.MPI_TAG);
        }
    }
    else{
        /* Send a message to the rank 0 process. The tag is the rank+100 */
        sprintf(message, "Hello from rank %d", myRank);
        MPI_Send(&message, len, MPI_CHAR, 0, myRank+100, MPI_COMM_WORLD);
    }

    MPI_Finalize();

    return 0;
}
```

Wildcards

Tag

wildcard.c

The rank 0 process can receive the messages in any order:

```
> mpicc -std=c99 -o wildcard wildcard.c
> mpirun -np 8 ./wildcard
Received message: "Hello from rank 4" from rank 4 with tag 104
Received message: "Hello from rank 1" from rank 1 with tag 101
Received message: "Hello from rank 5" from rank 5 with tag 105
Received message: "Hello from rank 6" from rank 6 with tag 106
Received message: "Hello from rank 7" from rank 7 with tag 107
Received message: "Hello from rank 2" from rank 2 with tag 102
Received message: "Hello from rank 3" from rank 3 with tag 103
>
> mpirun -np 8 ./wildcard
Received message: "Hello from rank 2" from rank 2 with tag 102
Received message: "Hello from rank 1" from rank 1 with tag 101
Received message: "Hello from rank 3" from rank 3 with tag 103
Received message: "Hello from rank 4" from rank 4 with tag 104
Received message: "Hello from rank 5" from rank 5 with tag 105
Received message: "Hello from rank 6" from rank 6 with tag 106
Received message: "Hello from rank 7" from rank 7 with tag 107
>
```

Groups and communicators

- We only need to collect results from the worker processes and not the manager process
- Instead of using the `MPI_COMM_WORLD` communicator we can define a new communicator which only includes the worker processes
- Define a group of processes which is a sub-set of `MPI_COMM_WORLD` and associate a new communicator with the group
- The new communicator can then be used instead of `MPI_COMM_WORLD` in MPI function calls

Typical process for creating a new communicator

1. Get a handle for the group associated with `MPI_COMM_WORLD` (`MPI_Comm_group`)
2. Make a new group from the world group (e.g. `MPI_Group_incl`)
3. Make a new communicator from the new group (`MPI_Comm_create`)
4. Find rank in new communicator (`MPI_Comm_rank`)
5. Do work and carry out communication
6. Free the communicator and group (`MPI_Comm_free` and `MPI_Group_free`)

Defining groups

<code>MPI_Group_incl</code>	Make a new group from an existing group including specified processes
<code>MPI_Group_excl</code>	Make a new group from an existing group excluding specified processes
<code>MPI_Group_union</code>	All elements in the first group followed by all elements in the second group not in the first group
<code>MPI_Group_intersection</code>	All elements in the first group that are also in the second group
<code>MPI_Group_difference</code>	All processes in the first group which are not in the second group

communicators.c

```
#include<stdio.h>
#include<mpi.h>

int main(int argc, char *argv[]){

    MPI_Group worldGroup, newGroup;
    MPI_Comm newComm;
    int ranks[4]={4,5,6,7};
    int myRank, nProcs;
    int myRankNew, nProcsNew;

    MPI_Init(&argc, &argv);

    /* Find the rank of this process in the world group */
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
    /* Find the total number of processes in the world group */
    MPI_Comm_size(MPI_COMM_WORLD, &nProcs);

    /* Get a group handle for the world group */
    MPI_Comm_group(MPI_COMM_WORLD, &worldGroup);
    /* Form a new group with the specified ranks */
    MPI_Group_incl(worldGroup, 4, ranks, &newGroup);
    /* Create a communicator from the new group */
    MPI_Comm_create(MPI_COMM_WORLD, newGroup, &newComm);

    if (myRank>3 && myRank <8){
        /* Find the rank of this process in the new group */
        MPI_Comm_rank(newComm, &myRankNew);
        /* Find the total number of processes in the new group */
        MPI_Comm_size(newComm, &nProcsNew);
        printf("I am process %d of %d in the world group and process %d of %d in the new group\n", \
            myRank, nProcs, myRankNew, nProcsNew);
    }

    MPI_Finalize();

    return 0;
}
```

Note: run with 8 MPI
processes

communicators.c

```
> mpicc -o communicators communicators.c
> mpirun -np 8 ./communicators
I am process 5 of 8 in the world group and process 1 of 4 in the new group
I am process 6 of 8 in the world group and process 2 of 4 in the new group
I am process 7 of 8 in the world group and process 3 of 4 in the new group
I am process 4 of 8 in the world group and process 0 of 4 in the new group
>
```

MPI_COMM_WORLD

newComm

Unit summary

- In this unit we have seen two additional MPI features
 - Wildcards: receive messages from any rank with any tag
 - Groups and communicators: define a communicator which is a sub-set of `MPI_COMM_WORLD`